Open CASCADE Technology
7.4.0

Contribution Workflow

September 27, 2019

Contents

1 Introduction **2**
1.1 Use of issue tracker system 2
1.2 Access levels 2

2 Standard workflow for an issue **3**
2.1 General scheme 3
2.2 Issue registration 4
2.3 Assigning the issue 5
2.4 Resolving the issue 5
2.4.1 Requirements to the code modification 5
2.4.2 Providing a test case 6
2.4.3 Updating user and developer guides 6
2.4.4 Submission of change as a Git branch 6
2.4.5 Requirements to the commit message 7
2.4.6 Marking issue as resolved 7
2.5 Code review 7
2.6 Testing 8
2.7 Integration of a solution 8
2.8 Closing an issue 9

3 Additional workflow elements **10**
3.1 Requesting more information or specific action 10
3.2 Defining relationships between issues 10
3.3 Submission of a change as a patch 10
3.4 Updating branches in Git 10
3.5 Minor corrections 11
3.6 Handling non-reproducible issues 11

4 Appendix: Issue attributes **12**
4.1 Category 12
4.2 Severity 12
4.3 Status 13
4.4 Resolution 13

(c) Open CASCADE 2019

1 Introduction

The purpose of this document is to describe standard workflow for processing contributions to certified version of OCCT.

1.1 Use of issue tracker system

Each contribution should have corresponding issue (bug, or feature, or integration request) registered in the Mantis↩ BT issue tracker system accessible by URL https://tracker.dev.opencascade.org. The issue is processed according to the described workflow.

1.2 Access levels

Access level defines the permissions of the user to view, register and modify issues in the issue tracker. The correspondence of access level and user permissions is defined in the table below.

Access level	Granted to	Permissions	Can set statuses
Viewer	Everyone (anonymous access)	View public issues only	None
Updater	Users registered on dev.↩ opencascade.org, in Open CASCADE project	View and comment issues	None
Reporter	Users registered on dev.↩ opencascade.org, in Community project	View, report, and comment issues	New, Resolved, Feedback
Developer	OCC developers and (in Community project) external contributors who signed the CLA	View, report, modify, and handle issues	New, Assigned, Resolved, Reviewed, Feedback
Tester	OCC engineer devoted to certification testing	View, report, modify, and handle issues	Assigned, Tested, Feedback
Maintainer	Person responsible for a project or OCCT component	View, report, modify, and handle issues	New, Resolved, Reviewed, Tested, Closed, Feedback
Bugmaster	Person responsible for Mantis issue tracker, integrations, certification, and releases	Full access	All statuses

According to his access level, the user can participate in the issue handling process under different roles, as described below.

2 Standard workflow for an issue

2.1 General scheme

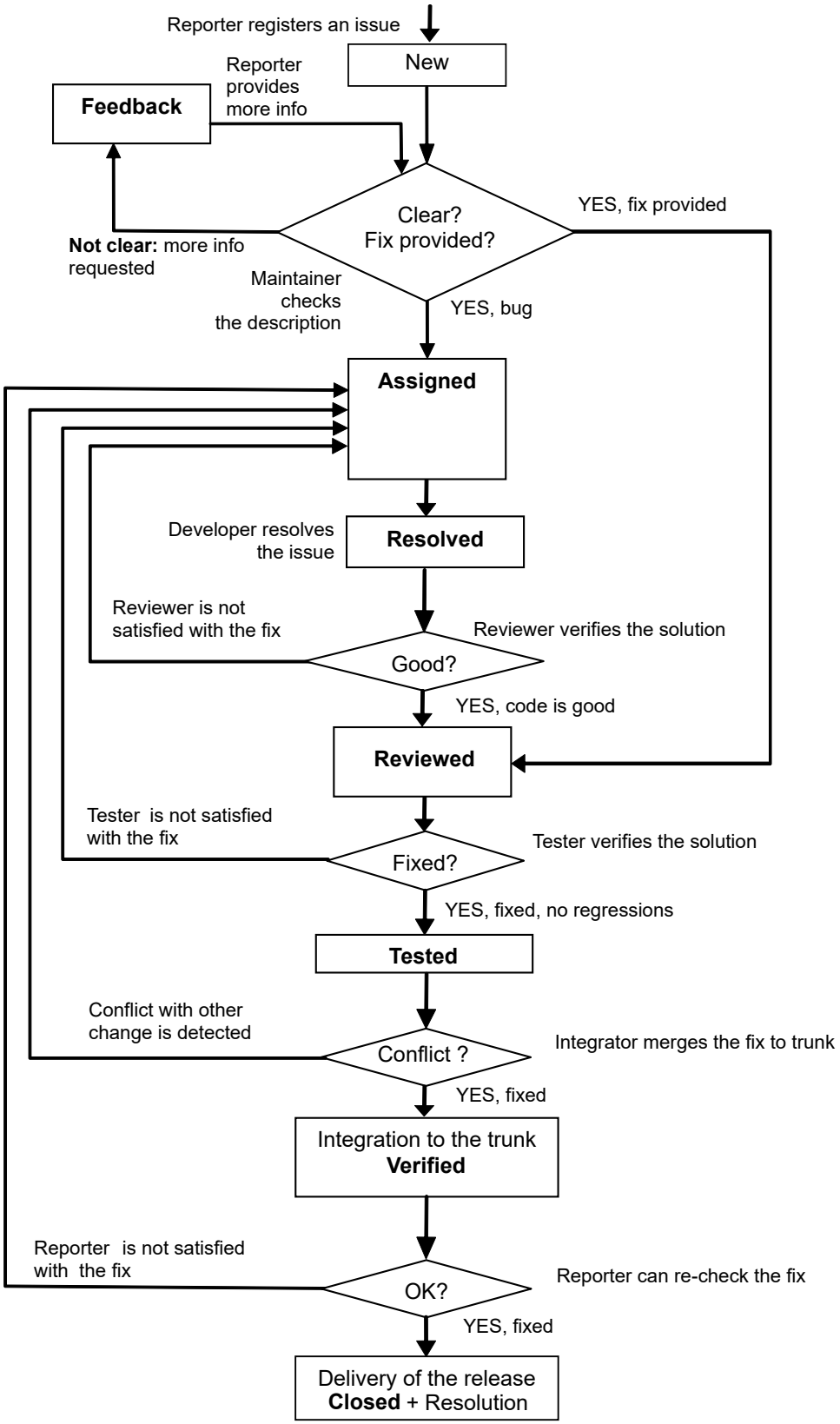

Figure 1: Standard life cycle of an issue

2.2 Issue registration

An issue is registered in Mantis bugtracker by the **Reporter** with definition of the necessary attributes (see also Appendix: Issue attributes):

Category – indicates the OCCT component, to which the issue relates. (If in doubt, assign to OCCT:Foundation Classes.)

Severity – indicates the impact of the issue in the context where it was discovered.

Profile – specifies the configuration, on which the problem was detected. For specific configurations it is possible to specify separately platform, OS, and version. These fields can be left empty if the issue is not configuration-specific. Additional details relevant for the environment where the issue is reproduced (such as compiler version, bitness, etc.) can be provided in the **Description**.

Products Version – defines the OCCT version, on which the problem has been detected.

It is preferable to indicate the version of the earliest known official release where the problem can be reproduced. If the issue is reported on the current development version of OCCT, the current development version should be used (for convenience, this version is marked by asterisk in Mantis).

Note

> OCCT version number can be consulted in the file Standard_Version.hxx (value of OCC_VERSION_COMP↩LETE macro).

Assign to – developer to whom the issue will be assigned. By default, it is set to **Maintainer** of the OCCT component selected in **Category** field.

Target Version – defines the target version for the fix to be provided. By default, it is set to the current version under development.

Summary – a short, one sentence description of the issue.

The **Summary** has a limit of 128 characters. It should be informative and useful for the developers. It is not allowed to mention the issue originator, and in particular the customer, in the name of the registered issue.

A good practice is to start the issue with indication of the relevant component (OCCT module, package, class etc.) to better represent its context.

The summary should be given in imperative mood when it can be formulated as goal to be achieved or action to be done. In particular, this applies to feature requests and improvements, for instance:

> *Visualization - provide a support of zoom persistent selection*

If the issue reports a problem, the summary should be given in Present Simple. If reported problem is believed to be a regression, it is recommended to indicate this in the summary, like this:

> [Regression in 6.9.0] *IGES - Export of a reversed face leads to wrong data*

Description – should contain a detailed definition of the nature of the registered issue depending on its type.

For a bug it is required to submit a detailed description of the incorrect behavior, including the indication of the cause of the problem (if known at this stage), and details on the context where the issue has been detected.

For a feature or integration request it is necessary to describe the proposed feature in details (as much as possible at that stage), including the changes required for its implementation and the main features of the new functionality.

Example:

> *Currently selection does not work correctly for non-zoomable objects (those defined using transform persistence). To provide correct selection for such objects, first-level (object) BVH structures must be updated on each camera change, and frustum must be rebuilt accordingly.*

Note

In the description and notes to the issues you can refer to another issue by its ID prefixed by number sign (e.g.: #12345), and refer to a note by its ID prefixed by tilde (e.g.: ∼20123). These references will be expanded by Mantis into links to the corresponding issue or note. When the number sign or the tilde followed by digits are a part of a normal text, add a space before digits (e.g.: "face # 12345 contains ∼ 1000 edges") to avoid this conversion.

Steps To Reproduce – allows describing in detail how to reproduce the issue.

This information is crucial for the developer to investigate the cause of the problem and to create the test case. The optimal approach is to give a sequence of DRAW Test Harness commands to reproduce the problem in DRAW. This information can also be provided as a DRAW Tcl script attached to the issue (in **Upload File** field).

Additional information and documentation updates – any additional information, remarks to be taken into account in Release Notes, etc..

Upload File – allows attaching the shapes, snapshots, scripts, documents, or modified source files of OCCT.

This field can be used to attach a prototype test case in form of a Tcl script for DRAW, a C++ code which can be organized in DRAW commands, sample shapes, documents describing proposed change or analysis of the problem, or other data required for reproduction of the issue. Where applicable, pictures demonstrating a problem and/or desired result can be attached.

The newly registered issue gets status **NEW** and is assigned to the person indicated in the **Assign to** field.

2.3 Assigning the issue

The description of the new issue is checked by the **Maintainer** and if it is feasible, he may assign the issue to a **Developer**. Alternatively, any user with **Developer** access level or higher can assign the issue to himself if he wants to provide a solution.

The recommended way to handle contributions is that the **Reporter** assigns the issue to himself and provides a solution.

The **Maintainer** or **Bugmaster** can close or reassign the issue (in **FEEDBACK** state) to the **Reporter** after it has been registered, if its description does not contain sufficient details to reproduce the bug or explain the need of the new feature. That decision shall be documented in the comments to the issue in the Bugtracker.

The assigned issue has status **ASSIGNED**.

2.4 Resolving the issue

The **Developer** responsible for the issue assigned to him provides a solution including:

- Changes in the code, with appropriate comments;
- Test case (when applicable) and data necessary for its execution;
- Changes in the user and developer guides (when necessary).

The change is integrated to branch named CRxxxxx (where **xxxxx** is issue number) in the OCCT Git repository, based on current master, and containing a single commit with the appropriate description. Then the issue is switched to **RESOLVED** for further review and testing.

The following sub-sections describe this process, relevant requirements and options, in more details.

2.4.1 Requirements to the code modification

The amount of code affected by the change should be limited to the changes required for the bug fix or improvement. Change of layout or re-formatting of the existing code is allowed only in the parts where meaningful changes related to the issue have been made.

Note

If deemed useful, re-formatting or cosmetic changes affecting considerable parts of the code can be made within a dedicated issue.

The changes should comply with the OCCT Codng Rules. It is especially important to comment the code properly so that other people can understand it easier.

The modification should be tested by running OCCT tests (on the platform and scope available to **Developer**) and ensuring absence of regressions. In case if modification affects results of some existing test case and the new result is correct, such test case should be updated to report OK (or BAD), as descibed in Automated Test System / Interpretation of Test Results.

2.4.2 Providing a test case

For modifications affecting OCCT functionality, a test case should be created (unless already exists) and included in the commit or patch. See Automated Test System / Creating a New Test for relevant instructions.

The data files required for a test case should be attached to the corresponding issue in Mantis (i.e. not included in the commit).

When the test case cannot be provided for any reason, the maximum possible information on how the problem can be reproduced and how to check the fix should be provided in the **Steps to Reproduce** field of an issue.

2.4.3 Updating user and developer guides

If the change affects a functionality described in User Guides, the corresponding user guide should be updated to reflect the change.

If the change affects OCCT test system, build environment, or development tools described in Developer Guides, the corresponding guide should be updated.

The changes that break compatibility with the previous versions of OCCT (i.e. affecting API or behavior of existing functionality in the way that may require update of existing applications based on an earlier official release of OCCT to work correctly) should be described in the document Upgrade from previous OCCT versions. It is recommended to add a sub-section for each change described. The description should provide the explanation of the incompatibility introduced by the change, and describe how it can be resolved (at least, in known situations). When feasible, the automatic upgrade procedure (adm/upgrade.tcl) can be extended by a new option to perform the required upgrade of the dependent code automatically.

2.4.4 Submission of change as a Git branch

The modification of sources should be provided in the dedicated branch of the official OCCT Git repository.

The branch should contain a single commit, with the appropriate commit message (see Requirements to the commit message below).

In general, this branch should be based on the recent version of the master branch. It is highly preferable to submit changes basing on the current master. In case if the fix is implemented on the previous release of OCCT, the branch can be based on the corresponding tag in Git, instead of the master.

The branch name should be composed of letters **CR** (abbreviation of "Change Request") followed by the issue ID number (without leading zeros). It is possible to add an optional suffix to the branch name after the issue ID, e.g. to distinguish between several versions of the fix (see Updating branches in Git).

See Guide to using GIT for help.

Note

When a branch with the name given according to the above rule is pushed to Git, a note is automatically added to the corresponding issue in Mantis, indicating the person who has made the push, the commit hash, and (for new commits) the description.

2.4.5 Requirements to the commit message

The commit message posted in Git constitutes an integral part of both the fix and the release documentation.

The first line of the commit message should contain the Summary of the issue (starting with its ID followed by colon, e.g. "0022943: Bug in TDataXtd_PatternStd"), followed by an empty line.

The following lines should provide a description of the context and details on the changes made. The contents and the recommended structure of the description depend on the nature of the bug.

In a general case, the following elements should be present:

- **Problem** – a description of the unwanted behavior;
- **Change** – a description of the implemented changes, including the names of involved classes / methods / enumerations etc.;
- **Result** – a description of the current behavior (after the implementation).

Example:

> *0026330: BRepOffsetAPI_ThruSections creates invalid shape.*
>
> *Methods BRep_Tool::CurveOnSurface() and BRepCheck_Edge::InContext() now properly handle parametric range on a 3D curve when it is used to generate a p-curve dynamically (on a planar surface) and both the surface and the 3D curve have non-null locations.*

Provide sufficient context so that potential user of the affected functionality can understand what has been changed and how the algorithm works now. Describe reason and essence of the changes made, but do not go too deep into implementation details – these should be reflected in comments in the code.

2.4.6 Marking issue as resolved

To mark the change as ready for review and testing, the corresponding issue should be switched to **RESOLVED** state. By default, the issue gets assigned to the **Maintainer** of the component, who is thus responsible for its review. Alternatively, another person can be selected as a reviewer at this step.

When the issue is switched to **RESOLVED**, it is required to update or fill the field **Steps to reproduce**. The possible variants are:

- The name of an existing or new test case (preferred variant);
- A sequence of DRAW commands;
- N/A (Not required / Not possible / Not applicable);
- Reference to an issue in the bug tracker of another project.

2.5 Code review

The **Reviewer** analyzes the proposed solution for applicability in accordance with OCCT Coding Rules and examines all changes in the sources, test case(s), and documentation to detect obvious and possible errors, misprints, or violations of the coding style.

If the Reviewer detects some problems, he can either:

- Fix these issues and provide a new solution. The issue can then be switched to **REVIEWED**.

  In case of doubt or possible disagreement the **Reviewer** can reassign the issue (in **RESOLVED** state) to the **Developer**, who then becomes a **Reviewer**. Possible disagreements should be resolved through discussion, which is done normally within issue notes (or on the OCCT developer's forum if necessary).

- Reassign the issue back to the **Developer**, providing detailed list of remarks. The issue then gets status **ASSIGNED** and a new solution should be provided.

If Reviewer does not detect any problems, or provides a corrected version, he changes status to **REVIEWED**. The issue gets assigned to the **Bugmaster**.

2.6 Testing

The issues that are in **REVIEWED** state are subject of certification (non-regression) testing. The issue is assigned to an OCCT **Tester** when he starts processing it.

If the branch submitted for testing is based on obsolete status of the master branch, **Tester** rebases it on master HEAD. In case of conflicts, the issue is assigned back to **Developer** in **FEEDBACK** status, requesting for a rebase.

Certification testing includes:

- Addition of new data models (if required for a new test case) to the data base;
- Revision of the new test case(s) added by developer, and changes in the existing test cases included in commit. The **Tester** can amend tests to ensure their correct behavior in the certification environment.
- Building OCCT on a sub-set of supported configurations (OS and compiler), watching for errors and warnings;
- Execution of tests on sub-set of supported platforms (at least, one Windows and one Linux configuration), watching for regressions;
- Building OCCT samples, watching for errors;
- Building and testing of OCC products based on OCCT.

If the **Tester** does not detect problems or regressions, he changes the status to **TESTED** for further integration.

If the **Tester** detects build problems or regressions, he changes the status to **ASSIGNED** and reassigns the issue to the **Developer** with a detailed description of the problems. The **Developer** should analyze the reported problems and, depending on results of this analysis, either:

- Confirm that the detected problems are expected changes and they should be accepted as a new status of the code. Then the issue should be switched to **FEEDBACK** and assigned to the **Bugmaster**.
- Produce a new solution (see Resolving the issue, and also Minor corrections).

2.7 Integration of a solution

Before integration into the master branch of the repository the **Integrator** checks the following conditions:

- the change has been reviewed;
- the change has been tested without regressions (or with regressions treated properly);
- the test case has been created for this issue (when applicable), and the change has been rechecked on this test case;
- the change does not conflict with other changes integrated previously.

If the result of check is successful the Integrator integrates the solution into the branch. The integrations are performed weekly; integration branches are named following the pattern IR-YYYY-MM-DD.

Each change is integrated as a single commit without preserving the history of changes made in the branch (by rebase, squashing all intermediate commits if any), however, preserving the author when possible. This is done to have the master branch history plain and clean. The following picture illustrates the process:

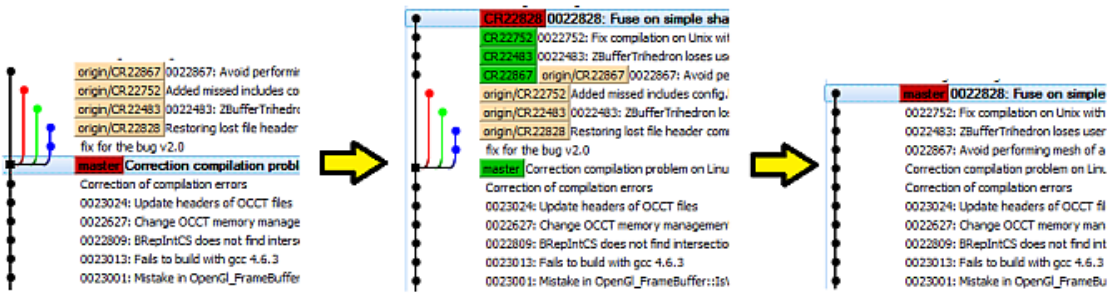

Figure 2: Integration of several branches

The new integration branch is tested against possible regressions that might appear due to interference between separate changes. When the tests are OK, the integration branch is pushed as the new master to the official repository. The issue status is set then to **VERIFIED** and is assigned to the **Reporter** so that he could check the fix as integrated.

The branches corresponding to the integrated fixes are removed from the repository by the **Bugmaster**.

2.8 Closing an issue

When possible, the **Reporter** should check whether the problem is actually resolved in the environment where it has been discovered, after the fix is integrated to master. If the fix does not actually resolve the original problem, the issue in **VERIFIED** status can be reopened and assigned back to the **Developer** for rework. The details on how to check that the issue is still reproducible should be provided. However, if the issue does resolve the problem as described in the original report, but a similar problem is discovered for another input data or configuration, or the fix has caused a regression, that problem should be registered as a separate (related) issue.

If the fix integrated to master causes regressions, **Bugmaster** can revert it and reopen the issue.

The **Bugmaster** closes the issue after the regular OCCT Release, provided that the issue status is **VERIFIED** and the change was actually included in the release. The final issue state is **CLOSED**.

The field **Fixed in Version** of the issue is set to the OCCT version where it is fixed.

3 Additional workflow elements

3.1 Requesting more information or specific action

If, at any step of the issue lifetime, the person responsible for it cannot process it due to absence of required information, expertise, or rights, he can switch it to status **FEEDBACK** and assign to the person who is (presumably) able to resolve the block. Some examples of typical situations where **FEEDBACK** is used are:

- The **Maintainer** or the **Developer** requests for more information from the **Reporter** to reproduce the issue;
- The **Tester** requests the **Developer** or the **Maintainer** to help him in the interpretation of testing results;
- The **Developer** or the **Maintainer** asks the **Bugmaster** to close the issue that is found irrelevant or already fixed (see Handling non-reproducible issues).

In general, issues with status **FEEDBACK** should be processed as fast as possible, to avoid unjustified delays.

3.2 Defining relationships between issues

When two or more issues are related to each other, this relationship should be reflected in the issue tracker. It is also highly recommended to add a note to explain the relationship. Typical cases of relationships are:

- Issue A is caused by previous fix made for issue B (A is a child of B);
- Issue A describes the same problem as issue B (A is a duplicate of B);
- Issues A and B relate to the same piece of code, functionality etc., in the sense that the fix for one of these issues will affect the other (A is related to B)

When the fix made for one issue also resolves another one, these issues should be marked as related or duplicate. In general, the duplicate issue should have the same status, and, when closed, be marked as fixed in the same OCCT version, as the main one.

3.3 Submission of a change as a patch

In some cases (if Git is not accessible for the contributor), external contributions can be submitted as a patch file (generated by *diff* command) or as modified sources attached to the Mantis issue. The OCCT version, for which the patch is generated, should be clearly specified (e.g. as hash code of Git commit if the patch is based on an intermediate state of the master).

Note

Such contributions should be put to Git by someone else (e.g. the **Reviewer**), this may cause delay in their processing.

3.4 Updating branches in Git

Updates of the existing branch (e.g. taking into account the remarks of the **Reviewer**, or fixing regressions) should be provided as new commits on top of previous state of the branch.

It is allowed to rebase the branch on the new state of the master and push it to the repository under the same name (with *–force* option) provided that the original sequence of commits is preserved.

When a change is squashed into a single commit (e.g. to be submitted for review), it should be pushed into a branch a with different name.

The recommended approach is to add a numeric suffix (index) indicating the version of the change, e.g. "CR12345↩ _5". Usually it is worth keeping a non-squashed branch in Git for reference.

To avoid confusions, the branch corresponding to the latest version of the change should have a greater index.

Note

Make sure to revise the commit message after squashing a branch, to keep it meaningful and comprehensive.

3.5 Minor corrections

In some cases review remarks or results of testing require only minor corrections to be done in the branch containing a change. "Minor" implies that the correction does not impact the functionality and does not affect the description of the previously committed change.

As an exception to general single-commit rule, it is allowed to put such minor corrections on top of the existing branch as a separate commit, and re-submit it for further processing in the same branch, without squashing.

Minor commits should have a single-line message starting with #. These messages will be ignored when the branch is squashed at integration.

Typical cases of minor corrections are:

- Amendments of test cases (including those made by the **Tester** to adjust a test script to a specific platform);
- Trivial corrections of compilation warnings (such as removal of an unused variable);
- Addition or correction of comments or documentation;
- Corrections of code formatting (e.g. reversion of irrelevant formatting changes made in the main commit).

3.6 Handling non-reproducible issues

Investigation of each issue starts with reproducing it on current development version (master).

If it cannot be reproduced on the current master, but can be reproduced on one of previous releases (or previous state of the master), it is considered as solved by a change made for another issue. If that "fixing" issue can be identified (e.g. by parsing Git log), it should be set as related to that issue. The issue should be switched to **FEEDBACK** and assigned to the **Bugmaster** for further processing.

The **Bugmaster** decides whether it is necessary to create a test case for that issue, and if so may assign it to the **Developer** or the **Tester** to create a test. The issue then follows the standard workflow.

Otherwise, if the issue is fixed in one of previous releases, the **Bugmaster** closes it setting the appropriate value in **Fixed in Version** field, or, if the issue is fixed after the last release, switches it to **VERIFIED** status.

If the issue cannot be reproduced due to an unclear description, missing data, etc., it should be assigned back to the **Reporter** in **FEEDBACK** status, requesting for more information. The **Reporter** should provide additional information within one month; after that time, if no new information is provided, the issue should be closed by the **Bugmaster** with resolution **Unable to reproduce**.

4 Appendix: Issue attributes

4.1 Category

The category corresponds to the component of OCCT where the issue is found:

Category	Component
OCCT:Foundation Classes	Foundation Classes module
OCCT:Modeling Data	Modeling Data classes
OCCT:Modeling Algorithms	Modeling Algorithms, except shape healing and meshing
OCCT:Shape Healing	Shape Healing component (TKShapeHealing)
OCCT:Mesh	BRepMesh algorithm
OCCT:Data Exchange	Data Exchange module
OCCT:Visualization	Visualization module
OCCT:Application Framework	OCAF
OCCT:DRAW	DRAW Test Harness
OCCT:Tests	Automatic Test System
OCCT:Documentation	Documentation
OCCT:Coding	General code quality
OCCT:Configuration	Configuration, build system, etc.
OCCT:Releases	Official OCCT releases
Website:Tracker	OCCT Mantis issue tracker, tracker.dev.opencascade.org
Website:Portal	OCCT development portal, dev.opencascade.org
Website:Git	OCCT Git repository, git.dev.opencascade.org

4.2 Severity

Severity shows at which extent the issue affects the product. The list of used severities is given in the table below in the descending order.

Severity	Description
crash	Crash of the application or OS, loss of data
block	Regression corresponding to the previously delivered official version. Impossible operation of a function on any data with no work-around. Missing function previously requested in software requirements specification. Destroyed data.
major	Impossible operation of a function with existing work-around. Incorrect operation of a function on a particular dataset. Impossible operation of a function after intentional input of incorrect data. Incorrect behavior of a function after intentional input of incorrect data.
minor	Incorrect behavior of a function corresponding to the description in software requirements specification. Insufficient performance of a function.
tweak	Ergonomic inconvenience, need of light updates.
text	Non-conformance of the program code to the Coding Rules, mistakes and non-functional errors in the source text (e.g. unnecessary variable declarations, missing comments, grammatical errors in user manuals).
trivial	Cosmetic issues.
feature	Request for a new feature or improvement.
integration request	Requested integration of an existing feature into the product.
just a question	A question to be processed, without need of any changes in the product.

(c) Open CASCADE 2019

4.3 Status

The bug statuses that can be applied to the issues are listed in the table below.

Status	Description
New	A new, just registered issue.
Acknowledged	Can be used to mark the issue as postponed.
Confirmed	Can be used to mark the issue as postponed.
Feedback	The issue requires more information or a specific action.
Assigned	Assigned to a developer.
Resolved	The issue has been fixed, and now is waiting for review.
Reviewed	The issue has been reviewed, and now is waiting for testing (or being tested).
Tested	The fix has been internally tested by the tester with success on the full non-regression database or its part and a test case has been created for this issue.
Verified	The fix has been integrated into the master of the corresponding repository
Closed + resolution	The fix has been integrated to the master. The corresponding test case has been executed successfully. The issue is no longer reproduced.

4.4 Resolution

Resolution is set when the bug is closed. "Reopen" resolution is added automatically when the bug is reopened.

Resolution	Description
Open	The issue is pending.
Fixed	The issue has been successfully fixed.
Reopened	The bug has been reopened because of insufficient fix or regression.
Unable to reproduce	The bug is not reproduced.
Not fixable	The bug cannot be fixed because e.g. it is a bug of third party software, OS or hardware limitation, etc.
Duplicate	The bug for the same issue already exists in the tracker.
Not a bug	It is a normal behavior in accordance with the specification of the product.
No change required	The issue didn't require any change of the product, such as a question issue.
Suspended	The issue is postponed (for Acknowledged status).
Documentation updated	The documentation has been updated to resolve a misunderstanding causing the issue.
Won't fix	It is decided to keep the existing behavior.